

Desenvolvimento de Aplicações Web

Lab 07 - Introdução ao Flask

Ano Letivo: 2025/2026

1 Introdução

Até agora, explorámos o lado visual das aplicações web — o **Front-end**. No entanto, uma aplicação real é mais do que apenas cores e botões; precisa também de um **Back-end** que nos permite realizar funções mais complexas e ainda obter os dados guardados na nossa base de dados.

É desta forma que conseguimos garantir a persistência de todas as customizações e dados com que o utilizador interage.

Neste laboratório vamos aprender como o **front-end** de uma aplicação web interage com o próprio **back-end** utilizando **Flask**, uma framework em **Python**. No final desta aula, a aplicação não será apenas um documento; será um sistema capaz de persistir informação e responder a pedidos.

1.1 Contexto histórico: A Partida de 1º de Abril

A história do **Flask** é, no mínimo, caricata. Em 2010, um programador chamado *Armin Ronacher* decidiu pregar uma partida de 1 de abril (o Dia das Mentiras) à comunidade de *Python*.

Ele criou um pequeno "*wrapper*" (embrulho) que juntava duas ferramentas que ele próprio já tinha desenvolvido: a **Werkzeug** (responsável pela comunicação com o servidor) e a **Jinja2** (que se foca no aspeto visual/templates).

Este projeto de brincadeira ganhou o nome de "*Denied*" e foi publicado como se fosse uma *framework* super minimalista que cabia num único ficheiro. O que *Armin* não esperava era que os programadores ficassem fascinados! Naquela altura, o *Django* era a ferramenta principal, mas muitos achavam-no demasiado pesado e rígido para projetos mais simples.

O **Flask** apareceu como um sopro de ar fresco: era leve, flexível e permitia que cada um construísse a aplicação à sua maneira. O sucesso foi tanto que a "mentira" tornou-se uma das *frameworks* mais utilizadas no mundo, provando que, às vezes, as melhores ideias surgem quando estamos apenas a divertir-nos.

2 Objetivos

Neste laboratório iremos:

- Aprender os conceitos básicos do Flask.
- Estruturar uma aplicação web com Front-end e Back-end.
- Integrar e persistir dados com uma base de dados em SQL Lite.

3 Pré-requisitos

- VSCode instalado
- Um browser moderno com ferramentas de desenvolvimento (Chrome, Firefox, Edge, Safari)
- Pasta `PaginaPessoal` dos laboratórios anteriores (Labs 01-06)
- Conhecimentos básicos de HTML, CSS e Python
- Noções gerais de programação (variáveis, condicionais, ciclos)

4 Conceitos-Chave

4.1 Flask: Uma framework para desenvolvimento Web em Python

Flask é uma micro-framework web desenvolvida em Python que permite criar aplicações web de forma simples e rápida.

Ao contrário de frameworks mais complexas, como Django, o Flask fornece apenas o essencial para criar aplicações web, permitindo ao programador adicionar apenas os componentes necessários, não impondo uma estrutura tão rígida como outras *frameworks*.

Características principais:

- **Leve e minimalista:** Inclui apenas funcionalidades essenciais
- **Baseado em Python:** Utiliza a sintaxe simples e clara da linguagem Python
- **Flexível:** Permite escolher bibliotecas externas conforme a necessidade
- **Sistema de rotas simples:** Associa URLs diretamente a funções Python
- **Motor de templates integrado:** Possibilita a utilização de templates para serem reutilizados para gerar páginas HTML dinâmicas

4.2 Estrutura MVC

Uma prática comum é seguir sempre um **padrão de design de software**, mesmo que a nossa aplicação seja pequena.

No futuro, se pretendermos adicionar algumas funcionalidades, será mais fácil fazê-lo se o nosso código seguir um padrão como o **MVC**, pois estará mais organizado e será mais fácil de manter e reutilizar.

Na imagem seguinte é possível observar como o **padrão MVC** pode ser **aplicado a uma aplicação web** desenvolvida em **Flask**:

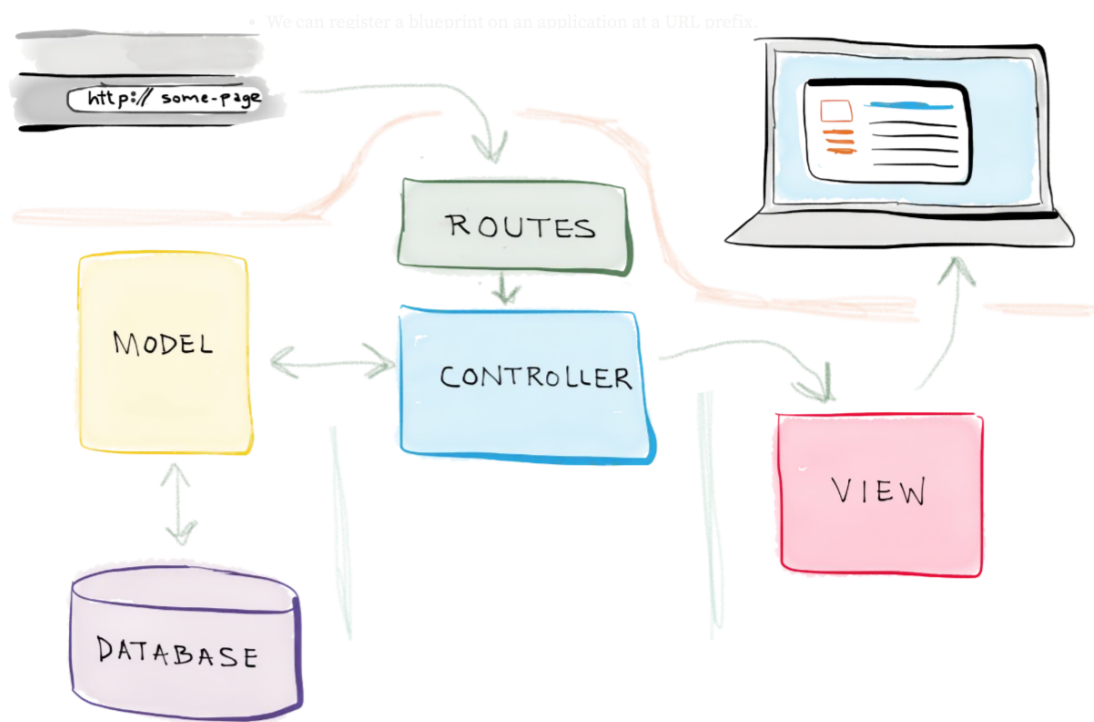


Figura 1: Modelo MVS com Flask

4.2.1 Rotas e URLs Dinâmicos

Quando um utilizador abre um website num browser, é realizado um pedido para que seja possível obter a informação necessária para renderizar a aplicação web. As routes são utilizadas em Flask como forma de encaminhar o pedido para o Controller.

As rotas são então os endereços (URLs) que a aplicação reconhece e podem ser:

- **Estáticas:** Como /home ou /pessoal.
- **Dinâmicas:** Como /movie/<int:movie_key>, permite que uma única função em Python trate milhares de páginas diferentes (cada uma para um filme específico) apenas alterando o ID no URL.

4.2.2 Controller

O Controller contém a lógica da aplicação que é responsável por gerir todos os pedidos realizados entre o front-end e o nosso back-end, da seguinte forma:

- Receber o pedido da rota
- Obter toda a informação necessária aos modelos
- Enviar os dados obtidos para as Views
- Reencaminhar toda a página HTML a renderizar

4.2.3 Model

O Model é responsável por toda a manutenção dos dados. Através da ligação com a nossa base de dados, permite a adição, leitura e edição de toda a informação que nela é persistida. É neste elemento deste padrão que são desenvolvidas todas as funções que pretendemos utilizar para a manipulação da informação.

4.2.4 View

Contêm toda a definição de lógica responsável pela seleção de que página HTML deve ser mostrada ao utilizador consoante a ação realizada. Após receber os dados que o Controller recebeu do Model, a View irá renderizar a nossa página HTML com esses mesmos dados, para que possam ser mostrados ao utilizador.

Resumo do fluxo MVC em Flask

1. O utilizador abre uma página no browser
2. A Route recebe o pedido
3. O Controller processa o pedido e solicita os dados
4. O Model vai buscar dados à Base de dados
5. O Controller envia os dados para a View
6. A View renderiza o resultado para o utilizador

4.3 Templates

Os *templates* permitem-nos escrever HTML com blocos reutilizáveis que são posteriormente preenchidos por funções de *Python*. Isto faz com que o código duplicado por toda a nossa aplicação seja cada vez menor, garantindo que apenas temos o código diferente presente em cada uma das nossas páginas.

4.3.1 Herança de Templates

Com o `{% extends "layout.html" %}`, evitamos repetir o menu e o rodapé em todas as páginas. Se **alterarmos o menu** no *layout*, o mesmo **é aplicado em toda a aplicação** automaticamente!

- **Separação de Preocupações:** O código *Python* torna-se responsável pela lógica (obter dados da BD), enquanto o ficheiro HTML foca-se na estrutura visual.
- **Reutilização de Código (Layouts):** Através da tag `{% extends "layout.html" %}`, podemos definir uma estrutura comum (menu, rodapé) e apenas adicionar o conteúdo específico de cada página.
- **Conteúdo Dinâmico:** Permite apresentar dados variáveis, como o dia da semana ou uma lista de filmes, utilizando a sintaxe `{{ variavel }}`.
- **Lógica no HTML:** Podemos usar ciclos `for` para gerar tabelas automaticamente com dados da base de dados e condicionais `if` para mostrar ou esconder elementos.

5 Procedimento

Neste laboratório, vamos estender a nossa aplicação para gerir uma coleção de filmes. Vamos começar por acrescentar à nossa aplicação de página pessoal, já criada anteriormente, uma página de filmes que contém uma lista dos nossos filmes favoritos.

5.1 Preparar o projeto com Flask

1. Instale a extensão *Python* disponível no VS Code.
2. No terminal, execute a instalação do Flask através do seguinte comando:

```
pip install Flask
```

Código 1: *Instalação do Flask.*

3. Crie um ficheiro chamado *server.py* na pasta principal do projeto. Este ficheiro é responsável pela definição de que rotas devem ser utilizadas para cada página HTML.

```
# Importação do pacote Flask
from flask import Flask
import views

# Definição de rotas da aplicação
def create_app():
    # Instaciação da aplicação Flask
    app = Flask(__name__)

    # Definição de configurações de execução da aplicação
    app.config.from_object("settings")

    # Definição da rota para a nossa página principal
    app.add_url_rule("/", view_func=views.home_page)

    return app

if __name__ == "__main__":
    app = create_app()
    port = app.config.get("PORT", 5000)
    app.run(host="0.0.0.0")
```

Código 2: *Ficheiro server.py.*

4. Também na pasta de raiz do projeto, crie um novo ficheiro *settings.py*.

```
# Modo de debug ativo
DEBUG = True
# Porta de execução da aplicação
PORT = 8080
# Chave usada para proteger as sessões da aplicação
SECRET_KEY = "secret"
```

Código 3: *Ficheiro com configurações de execução da aplicação.*

Atenção!!

O modo **debug** pode causar diversos problemas de segurança para uma aplicação web. Por esta razão, este modo **deverá ser utilizado apenas durante o desenvolvimento**.

5. Vamos agora tratar de organizar a view que pretendemos chamar ao executar a nossa aplicação. Crie um ficheiro chamado `views.py` com o seguinte código:

```
from flask import render_template

def home_page():
    return render_template("index.html")
```

Código 4: Ficheiro de definição de views.

Cuidado

Neste momento **se tentarmos executar** a nossa aplicação, **não iremos conseguir!** Isto acontece porque, como já estamos a introduzir os conceitos de **Flask** é necessário respeitar a organização de pastas do mesmo, pois a nossa view procura executar o ficheiro `HTML` presente na pasta `templates`.

6. Vamos então criar uma pasta:
- (a) **templates** e mover as nossas **páginas HTML** para lá.
 - (b) **static** e mover as pastas `audio`, `images`, `scripts`, `styles` e `video`.

Neste momento, a sua organização de pastas deverá ter a seguinte apresentação:

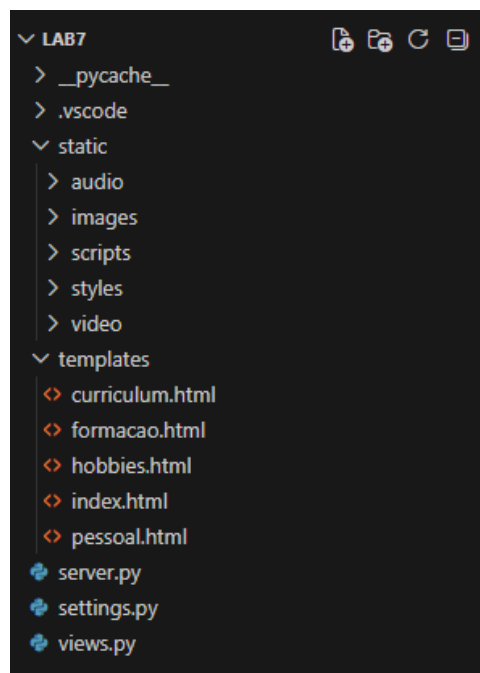


Figura 2: Organização de pastas até ao momento

7. Vamos então executar a nossa aplicação. Corra o seguinte código no terminal:

```
python server.py
```

Sem imagens e estilos?

Para que a nossa aplicação funcione corretamente, será necessário atualizar o caminho relativo dos objetos presentes em links, em todas as páginas. Para corrigir isto, teremos de **mudar** as referências *static* de *images/foto.jpg* para ***static/images/foto.jpg***. Deverá fazer o mesmo para os **vídeos, áudios, estilos e scripts**.

Nota: Neste momento apenas o link de menu do *index* deverá funcionar. Isto deve-se ao facto dos restantes ainda não terem as rotas configuradas. Trataremos disso nos próximos passos.

8. Para entendermos melhor a utilidade de uma *view*, vamos agora tentar adicionar uma saudação na nossa página inicial que introduz o dia da semana (calculado com base no próprio dia).

(a) Adicionar o seguinte código na definição da nossa *view*:

```
from flask import render_template
from datetime import datetime

# Necessário para apresentar os nomes dos dias em pt-pt
days_pt = [
    "Segunda-feira",
    "Terça-feira",
    "Quarta-feira",
    "Quinta-feira",
    "Sexta-feira",
    "Sábado",
    "Domingo"
]

def home_page():
    # Função para obter o dia de hoje
    today = datetime.today()
    # Extrai o nome do dia da semana
    day_name = days_pt[today.weekday()]
    # Enviamos o nome do dia como um input para o nosso template
    return render_template("index.html", day=day_name)
```

Código 5: Atualização da *view* da Home Page

(b) Adicionar o seguinte código no ficheiro *index.html*:

```
<p class="subtitle">Hoje é {{ day }}!</p>
```

(c) Volte a executar a aplicação. Deverá conseguir ver o dia de hoje na sua página.

9. Vamos agora criar um novo ficheiro denominado de *layout.html*. Este ficheiro deverá ser reutilizado por todas as nossas páginas *HTML*. Tenha em **atenção** que deverá **manter a ligação correta** com **os seus ficheiros de estilo e scripts**.

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width,
    initial-scale=1" />
  <title>Página Pessoal - [ SEU NOME ]</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='
    styles/style.css') }}">
  <script src="{{ url_for('static', filename='scripts/main.js')
    }}"></script>
</head>

<body>
  <header>
    <nav class="navbar" aria-label="main navigation">
      <ul>
        <li><a class="botao" href="{{ url_for('home_page')
          }}">Home</a></li>
        <li><a class="botao" href="pessoal.html">Dados
          Pessoais</a></li>
        <li><a class="botao" href="formacao.html">Formaçã
          o</a></li>
        <li><a class="botao" href="curriculum.html"
          >Curriculum</a></li>
        <li><a class="botao" href="hobbies.html"
          >Hobbies</a></li>
      </ul>
    </nav>

    {% block title %}{% endblock %}
  </header>

  <div class="content">
    {% block content %}{% endblock %}
  </div>

  <footer class="footer">
    <p>Copyright &copy; 2026 [ SEU NOME ]. Todos os direitos
      reservados.</p>
    <p>Contacto: [ EMAIL ]</p>
  </footer>
</body>

</html>
```

Código 6: Ficheiro layout.html

Porquê usar `url_for`?

Para garantir que os ficheiros **CSS**, **JavaScript**, imagens e outros recursos estáticos são sempre encontrados corretamente, é recomendado usar `{{ url_for('static', filename='...') }}` em vez de caminhos relativos. Desta forma, o *Flask* gera automaticamente o caminho correto para a pasta *static*, evitando erros quando a página muda de rota e facilitando a manutenção da aplicação.

10. Vamos agora reutilizar o nosso ficheiro de layout na página *index.html*.

```
{% extends "layout.html" %}
{% block title %}
    <!-- O vosso título do cabeçalho da página -->
<p class="subtitle">Hoje é {{ day }}!</p>
{% endblock %}
{% block content %}
    <!-- Todo o vosso código de conteúdo com semântica aqui -->
{% endblock %}
```

Código 7: Reutilização do template de layout.

11. Execute a aplicação novamente. Deverá ter exatamente a mesma estrutura e informação que anteriormente.
12. Neste momento os links para as restantes páginas não funcionam. Repita o seguinte processo de forma a colocar toda a navegação operacional:
- (a) Definir que página *HTML* a *view* deve abrir
 - (b) Adicionar o *url* correto no ficheiro *layout.html*
 - (c) Adicionar a *route* no ficheiro *server.py*
13. Crie agora uma página extra para apresentar uma lista de filmes que recomende e toda a navegação para lá. A página deverá ter apenas o título "Filmes Favoritos".

5.2 Persistência de dados

1. Vamos agora preencher nossa lista de filmes com dados de uma base de dados.
- (a) Comece por criar uma pasta *models* na raiz do projeto e crie um ficheiro no interior desta pasta denominado *movies.sqlite*. Este ficheiro representa a nossa BD.
 - (b) Precisamos agora de informar a nossa aplicação sobre qual é a nossa BD a usar no projeto, adicionando essa informação no ficheiro *server.py*:

```
def create_app():
    # ...
    # Adicionar a info da BD depois de todas as routes

    models_dir = os.path.dirname(os.path.abspath(__file__)) + "\
models"
    db = Database(os.path.join(models_dir, "movies.sqlite"))
    app.config["db"] = db

    return app
```

Código 8: Informação da BD a adicionar no *server.py*

(c) Crie o ficheiro *movie.py* na pasta *models*, com o seguinte código:

```
# Definição da classe Movie (filme)
class Movie:

    # Método construtor da classe, chamado automaticamente
    # quando é criado um novo objeto Movie
    def __init__(self, title, year=None):

        # Guarda o título do filme no objeto
        self.title = title

        # Guarda o ano do filme no objeto
        self.year = year
```

Código 9: Ficheiro *movie.py*.

(d) Crie o ficheiro *database.py* também na pasta *models*. É neste ficheiro que serão definidas as funções a executar na BD.

```
import sqlite3 as dbapi2
from models.movie import Movie

# Classe responsável por gerir a base de dados
class Database:

    # Construtor da classe
    def __init__(self, dbfile):
        self.dbfile = dbfile # Guarda o nome do ficheiro da
                               # base de dados
        self.create_table() # Chama o método para criar a
                             # tabela (se ainda não existir)

    # Método que cria a tabela MOVIE na base de dados
    def create_table(self):
        # Abre ligação 'abase de dados e garante que a ligação é
        # fechada automaticamente no final
        with dbapi2.connect(self.dbfile) as connection:
            cursor = connection.cursor() # Cria um cursor para
                                           # executar comandos SQL

            # Executa um comando SQL para criar a tabela se ela
            # ainda não existir
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS MOVIE (
                    ID INTEGER PRIMARY KEY AUTOINCREMENT,
                    TITLE VARCHAR(80) NOT NULL UNIQUE,
                    YEAR INTEGER
                )""")

            # Executa um comando SQL para inserir um filme na
            # tabela MOVIE
            # "INSERT OR IGNORE" significa que se já existir um
            # registo igual, o comando é ignorado
            cursor.execute("""INSERT OR IGNORE INTO MOVIE (TITLE
                , YEAR) VALUES (?, ?)""", ("The Matrix", 1999))
```

```

        cursor.execute("""INSERT OR IGNORE INTO MOVIE (TITLE
            , YEAR) VALUES (?, ?)""", ("Inception", 2010))

        # Guarda as alterações na base de dados
        connection.commit()

    # Método que vai buscar todos os filmes da base de dados
    def get_movies(self):
        movies = [] # Lista onde serão guardados os filmes

        # Abre ligação à base de dados
        with dbapi2.connect(self.dbfile) as connection:
            cursor = connection.cursor() # Cria cursor

            # Query SQL para seleccionar todos os filmes
            # ordenados pelo ID
            query = "SELECT ID, TITLE, YEAR FROM MOVIE ORDER BY
                ID"
            cursor.execute(query)

            # Percorre os resultados devolvidos pela base de
            # dados
            for movie_key, title, year in cursor:

                # Cria um objeto Movie com o título e o ano
                # Guarda na lista (ID, objeto Movie)
                movies.append((movie_key, Movie(title, year)))

        # Devolve a lista de filmes
        return movies

```

Código 10: Ficheiro *database.py*

- (e) No ficheiro *filmes.html*, adicione o seguinte código no *body*. Este código é responsável por renderizar linhas numa tabela para cada registo de filmes presente na BD.

```

{% if movies %}
<table>
    {% for movie_key, movie in movies %}
    <tr>
        <td>
            {{ movie.title }}
            {% if movie.year %} ({{ movie.year }}) {% endif %}
        </td>
    </tr>
    {% endfor %}
</table>
{% endif %}

```

Código 11: Ficheiro *filmes.html*.

- (f) Sendo que queremos passar uma lista de objetos para a nossa página, precisamos de definir essa lista na view da página.

```

from flask import current_app

def movies_page():

```

```
# Vai buscar o objeto da base de dados guardado na configura
ção da aplicação Flask
db = current_app.config["db"]

# Verifica se o pedido HTTP feito pelo utilizador é do tipo
GET
# (GET é normalmente usado para obter ou visualizar dados)
if request.method == "GET":

    # Vai buscar todos os filmes da base de dados
    movies = db.get_movies()

    # Renderiza (mostra) o template HTML "filmes.html" e
    envia a lista de filmes ordenados para o template
    return render_template("filmes.html", movies=sorted(
        movies))
```

Código 12: Renderização da view da página Filmes com acesso à BD.

- (g) Vamos voltar a executar a nossa aplicação. É suposto ver uma lista de 2 filmes na sua página *HTML*.
2. Em seguida, queremos adicionar uma página que irá apresentar apenas um filme. Para identificar um filme na coleção, iremos usar o valor do seu id como parte do URL. Por exemplo, a *route* deverá ser */movies/1* e irá referir-se ao filme com o id 1.

(a) Crie um ficheiro *filme.html* de forma a apresentar a informação de um filme.

```
{% extends "layout.html" %}
{% block title %}<h1 class="title">Filme</h1>{% endblock %}
{% block content %}
<table class="table">
  <tr>
    <th>Título:</th>
    <td>{{ movie.title }}</td>
  </tr>
  {% if movie.year %}
  <tr>
    <th>Ano:</th>
    <td>{{ movie.year }}</td>
  </tr>
  {% endif %}
</table>
{% endblock %}
```

Código 13: Ficheiro *filme.html*

3. Para navegar para esta nova página, vamos adicionar o link para a mesma no título do filme em *filmes.html*:

```
<a href="{{ url_for('movie_page', movie_key=movie_key) }}">{{ movie
.title }}</a>
```

Código 14: Link para o filme.

(a) Crie as restantes *routes* e *views* para que seja possível navegar para a nova página.

i. Criar função para a *view*.

```
def movie_page(movie_key):
    db = current_app.config["db"]

    # O movie_key é recebido a partir da URL (por exemplo: /
    # movie/123),
    # sendo automaticamente passado como argumento para esta
    # função.
    # Esse valor é depois usado no método get_movie para
    # procurar o filme na base de dados.
    movie = db.get_movie(movie_key)
    # Verifica se o filme não foi encontrado
    if movie is None:
        # Se não existir, devolve um erro HTTP 404 (Página n
        # ão encontrada)
        abort(404)
    return render_template("filme.html", movie=movie)
```

Código 15: Definição da *view* da página *filme.html*.

ii. Adicionar função para obter um filme no ficheiro *database.py*.

```
def get_movie(self, movie_key):
    with dbapi2.connect(self.dbfile) as connection:
        cursor = connection.cursor()
        query = "SELECT TITLE, YEAR FROM MOVIE WHERE (ID
        = ?)"
        cursor.execute(query, (movie_key,))
        title, year = cursor.fetchone()
        movie = Movie(title, year=year)
    return movie
```

Código 16: Obter um filme da *BD*.

iii. Acrescente a *route* para a nova página:

```
app.add_url_rule("/filme/<int:movie_key>", view_func=views.
movie_page)
```

Código 17: *Route* da *view* da página do filme.

(b) Teste a aplicação. Deverá conseguir entrar na página e verificar os dados individuais de cada filme.

6 Próximos Passos

No próximo laboratório (Lab 08), iremos aprofundar:

- Utilização de *Forms* para criação, edição e eliminação de informação.
- Utilização de *Logins* para controlo de acesso à aplicação.

Recursos recomendados para estudo:

- **MVC:** https://medium.com/@joespinelli_6190/mvc-model-view-controller-ef878e2fd6f5

7 Referências

- **Flask Application Development - Basics:** <https://web.itu.edu.tr/uyar/fad/basics.html>
- **Model View and Controller:** <https://medium.com/@clarazheng111/model-view-controller-5c1f16f23>
- **Flask forms and Validation:** <https://harshilbmk.medium.com/day-45-flask-forms-and-validation->